

NewsBot Intelligence System: Reflective Journal

Trilok Kalani | ITAI 2373 Natural Language Processing | Midterm Project | Group: Individual (solo) | GitHub:
[paste your repo link here]

Team Collaboration

I worked on this project by myself, so for me collaboration was less about splitting tasks between people and more about how I organized my own work across the eight modules. I planned it in the order the data actually flows. Preprocessing had to come first and had to be solid, because if that step is messy then the TF-IDF features and the classifier downstream end up messy too. Working alone actually made the links between the modules really clear, since I felt every handoff myself instead of just passing it to someone else. If I had been in a real group, I would have split the modules up by which ones depend on which, kept one shared notebook, and used GitHub branches so two of us were not overwriting the same code. The main thing I took away about teamwork on a project like this is that agreeing on the exact format of the data passed between steps matters more than who writes which function.

Technical Integration Challenges

The hardest part was not any single algorithm. It was getting eight different techniques to run together inside Colab's free tier without running out of time or memory. The spaCy parse was the slow piece, so I ended up using NLTK for the fast cleaning over the whole dataset and only running spaCy's heavier parsing on a sample. I also hit a bug that confused me at first: short or unusual inputs all came back as sport. After digging into it I realized that when none of the input words were in the model's vocabulary, the model just fell back to the most common category, which happened to be sport. I fixed it by having the system return uncertain instead of guessing, and that turned into one of my favorite parts because it is what a real tool should actually do. Another thing that surprised me was that combining my own engineered features with the TF-IDF features actually lowered the accuracy, which taught me you cannot just glue two feature sets together without thinking about scaling and weighting.

Business Value

It is not hard to see where this fits in the real world. A company's PR or communications team could run NewsBot on a news feed and have it automatically flag every article that mentions them, then tell them quickly if the coverage is starting to turn negative. The entity extraction basically turns a pile of articles into a searchable index of the people, companies, and places being talked about. I also tried to be honest about the limits instead of overselling it. The model is trained on older BBC articles, so it would do worse on messy, present-day news, and the sentiment part clearly misreads financial language, scoring a profit warning as positive. I would not trust that piece in a real product without replacing it first.

Individual Contributions

Because I did this on my own, I built every part of it: the preprocessing and TF-IDF feature extraction, the part-of-speech, syntax, sentiment, classification, and named entity recognition modules, the integration layer with the uncertainty guard, and the four bonus extensions. The

piece I am most proud of is an experiment I added myself to test whether the grammar and structure of an article alone, with the actual words stripped out, could still predict its category. It can, and it lands well above random chance, which I thought was a genuinely interesting result to find.

Future Enhancements

If I keep building on this, the first change I would make is swapping the simple sentiment lexicon for a model like FinBERT, since the sentiment mistakes were the most obvious weakness in the whole system. After that I would fine-tune the entity recognizer on more recent news so it stops mislabeling modern companies and places, add topic modeling that runs over time to track themes, and add support for languages other than English so it could be used outside English-only news.

Professional Development Impact

More than any one technique, this project taught me to think about a whole system instead of a single model. I had to design around a real compute limit, avoid data leakage by fitting the vectorizer only on the training split, compare several models fairly instead of just reporting the best one, and be upfront about what does not work. Those habits feel a lot closer to real engineering than a normal homework problem does. Being able to walk an interviewer through a project like this, from raw text all the way to a final prediction, is exactly the kind of thing I want in my portfolio, and it leaves me feeling more prepared for the deep learning, ROS, and capstone courses coming later in the program.